GeneStamp: Stealth-Tested Ownership Tagging for Engineered DNA

1. One-Line Pitch

A web platform that hides a short, error-corrected ownership ID inside natural-looking, function-preserving DNA, then proves the tag is hard to detect and hard to destroy using a built-in detector and an attack arena.

2. Origin

- Started from the spec "Steganographic DNA Storage via Adversarial Generative Architectures" (GAN generates DNA that looks like a real genome while carrying a payload; Reed-Solomon/LDPC fixes channel errors).
- Pivot: instead of covert messaging or bulk file storage, use the same machinery for **authenticity tagging of engineered strains, plasmids and synthetic genes**.
- Why the pivot: clear customer, clear threat model (someone trying to find and strip the tag), small payloads (much easier than file storage), and it avoids the "undetectable synthetic DNA" biosecurity objection.

3. Problem

- Biotech startups, university labs and seed/crop developers invest heavily in engineered constructs and strains.
- Stolen or leaked samples are hard to prove as theirs.
- Obvious tag sequences are easy to spot and cut out.
- Random inserted DNA can harm the host organism.
- Organisms mutate over generations, so a naive tag degrades.

4. What Already Exists (Prior Art)

Findings from a quick search; not a full literature review. Verify before citing in a paper.

- **Gene watermarking via steganography (PLOS ONE, 2012):** messages embedded in open reading frames of synthetic genes, adapting digital-media steganography. Closest classical idea to "hide the ID in codon choices".
- **Digital signatures for plasmids (Gallegos, Kar, Ray, Ray, Peccoud, ACS Synth. Biol., 2020):** pilot software that signs plasmids with a private key and verifies them. The authors note plain watermarks can be modified, removed or pasted into other sequences.
- **DNAMark and CentralMark (NeurIPS 2025):** watermarks for DNA produced by DNA language models. DNAMark uses synonymous codon substitutions with sparse watermarking; CentralMark makes the mark detectable in both DNA and the translated protein. Aimed at biosecurity of AI-designed sequences.
- **SynthID Bio (Google DeepMind, 2026):** watermarks AI-designed proteins by nudging amino acid choices inside ProteinMPNN, detected with a key. Reported weakness: re-running a marked protein through another design tool can erase the tag.

Common gaps in these works (as far as found):

- Mostly research papers or pilots, not a usable app for ordinary labs.
- Recent work targets AI-generated sequences and biosecurity, not tagging a lab's own constructs.
- Little emphasis on measured stealth against a trained detector.
- No interactive red-team or attack testing.
- No forensic "trace this sample back to its owner" workflow.

5. What Is New in This Project

Positioning: do **not** claim to have invented DNA watermarking. Claim the following.

1. **Adversarially trained realism:** a GAN discriminator plus k-mer MMD pushes stamped sequences toward the host genome's statistics, so a detector cannot tell them apart.
2. **Measured stealth:** report detector AUC (target near 0.5) against a naive-encoding baseline that gets caught easily, using a held-out detector never seen in training.
3. **Measured robustness:** an Attacker Arena where mutation, trimming, shuffling and recoding attacks run live against the stamp, producing a survival score.
4. **Function preservation as a proof, not a claim:** a side-by-side translation showing 100% protein match.
5. **A usable product layer:** Stamp, Verify, Stealth Check, Stress Lab, Forensic Trace and a registry in one deployed web app.

One-line differentiator: first usable, adversarially evaluated tagging platform for engineered DNA (to be checked against DNAMark in related work).

6. Product Features

1. **Stamp:** user enters an ID (company, project, batch) and pastes a coding sequence or plasmid region. App returns a modified sequence with the hidden ID, downloadable as FASTA, plus a proof panel (protein identical, GC content, codon usage similarity).
2. **Verify:** upload a suspect sequence (possibly mutated). App recovers the ID and reports errors corrected, or "no valid stamp found".
3. **Stealth Check:** detector scores the stamped sequence next to a naively tagged one. Naive is flagged, stamped is not.
4. **Stress Lab:** slider for substitution and indel rates (0.1% to 5%) with a live pass/fail on payload recovery.
5. **Attacker Arena:** pick an attack (random mutation, trimming, shuffling codons, re-synonymizing) and see whether the ID survives. Optional leaderboard.
6. **Forensic Trace ("Shazam for strains"):** upload an unknown sequence, search the registry, return the likely owner, confidence and suspected tampered positions.
7. **Evolution Simulator:** simulate 100 to 1000 generations of mutation and show the stamp decaying and ECC repairing it until it finally breaks.
8. **Registry:** log of each stamp with timestamp, owner and hash of the stamped sequence for ownership-dispute evidence.

7. Threat Model

Attacker goals:

- Find the stamp.
- Remove or corrupt the stamp without breaking the organism.
- Forge someone else's stamp.

Attacker capabilities assumed:

- Knows the algorithm (security must not depend on secrecy of the method).
- Has the stamped sequence and the host reference genome.
- Can run statistical detectors and recode synonymous codons.
- Can apply random mutations and trimming.

Defenses:

- Secret-key placement: the key decides which codons carry bits, so the stamp location is not findable from the algorithm alone.
- Natural-looking statistics: matched to host codon usage and k-mer profile.
- ECC plus redundancy: tolerates mutations and partial loss.
- Signed or MAC-protected ID: prevents forgery.

Known limit to state openly: an attacker who rewrites many synonymous codons across the whole gene can erase any codon-level stamp. The Arena exists to measure how much rewriting it takes, not to claim immunity.

8. Why Not Just Do the Spec As Written

Issues found in the original spec and how this project handles them.

- **Indels break the decoder:** the skeleton uses Gaussian noise on one-hot vectors and a mean-pooling decoder that throws away position. Real insertions and deletions shift the reading frame. Fix: model real substitution and indel noise, use synchronization markers or position-aware decoding, and let ECC handle residual errors.
- **Normalizing flows on discrete bases:** invertibility claims (G inverse equals decoder) do not hold directly on discrete A/C/G/T. Fix: drop flows for the MVP; use keyed synonymous-codon embedding as the encoder and use the GAN as a realism refiner.
- **Over-constraining biology:** forcing GC at 50% and homopolymers at 4 or less can look less natural than real genomes. Fix: match the host genome's local GC distribution instead of a fixed target.
- **Gumbel-softmax train/inference gap:** discriminator sees soft samples in training but hard sequences at inference. Fix: straight-through hard sampling in later epochs and evaluate only on hard sequences.
- **Weak evaluation:** a k-mer XGBoost detector alone is a weak test. Fix: also use a held-out CNN or small genomic transformer detector.
- **"Mathematically lossless":** only true while errors stay within ECC capacity. State the condition.

9. System Architecture (Text Description)

Stamp path:

1. Payload bits (ID plus short check value) go to the ECC encoder (Reed-Solomon).
2. Keyed embedder chooses synonymous codons in the input coding sequence to carry the coded bits. Protein stays identical.
3. Realism refiner (trained model) adjusts which synonymous choices are used so local statistics match the host genome.
4. Output sequence plus proof panel.

Verify path:

1. Suspect sequence enters the decoder, using the key to locate carrier positions.
2. Decoder estimates the codeword, tolerating substitutions and indel shifts.
3. ECC decoder corrects errors and returns the ID and check result.
4. Registry lookup for ownership and timestamp.

Evaluation path (separate from product): simulated channel, held-out detectors, attack suite.

10. ML Design

10.1 Components

- **Generator / embedder:** takes the codeword and a coding sequence and outputs synonymous-codon choices. Start with a conditional model over per-codon choices, not free generation of bases.
- **Discriminator (training-time):** 1D CNN over one-hot DNA windows, trained on real E. coli CDS windows versus stamped windows.
- **Neural decoder:** bidirectional LSTM or small transformer over the (noisy) sequence, outputting codeword bits. Must be position-aware.
- **Channel simulator:** substitutions, insertions, deletions at configurable rates (0.1% to 5%); optionally profile-based after the MVP (NanoSim or similar style).
- **Outer ECC:** Reed-Solomon (library-based), possibly with interleaving so bursts are spread.

10.2 Payload Design

- Target 64 to 128 raw bits.
- Suggested content: organization ID, project ID, batch number, short check value.
- Full digital signature is too large for the DNA payload; store a short MAC or hash in DNA and keep the full signature in the registry.

10.3 Loss Terms

- Adversarial loss: Wasserstein GAN with gradient penalty (realism).
- Reconstruction loss: cross-entropy between true codeword and decoder output after the simulated channel.
- Biological validity penalty: homopolymer runs above threshold, extreme local GC deviation, optional secondary-structure penalty.

- Statistical loss: maximum mean discrepancy between k-mer frequency vectors of generated and real sequences (k = 3 or 4).
- Function constraint: handled structurally (only synonymous codons allowed), not as a soft loss.

Total loss is a weighted sum of the four terms.

10.4 Training Schedule

1. **Phase 1, reconstruction warm-up (early epochs):** high weight on reconstruction, small biology weight, no adversarial term. Learn to carry and recover bits through the channel.
2. **Phase 2, biology and statistics:** add biological validity and k-mer MMD terms while keeping reconstruction weight.
3. **Phase 3, adversarial polish:** introduce the discriminator loss with moderate weight; monitor that bit recovery does not degrade.

- Use curriculum on channel noise: start with low noise, increase gradually.
- Track detector AUC and post-ECC error rate every few epochs; stop at the best trade-off, not the lowest loss.
- Exact weights are starting points to tune, not fixed values.

10.5 Data

- Host reference: E. coli K-12 MG1655 genome and annotated coding sequences from NCBI.
- Codon usage table computed from those coding sequences (or taken from a public codon usage database).
- Training windows: sampled real CDS segments; test on genes held out entirely.
- Optional second host (for example yeast) later to show generality.

10.6 Baselines to Compare Against

- Naive codebook encoding (fixed mapping of bits to bases, no realism).
- Simple synonymous-codon steganography without GAN refinement (ablation, shows what the GAN adds).
- Full model.

11. Evaluation Plan

- **Payload recovery:** post-ECC bit error rate and full-ID recovery rate across noise levels (0.1% to 5%), including indels.
- **Stealth:** AUC of (a) k-mer feature classifier and (b) held-out CNN or transformer detector. Target near 0.5 for the full model, near 1.0 for the naive baseline.
- **Function preservation:** protein identity check (must be 100% by construction) and codon adaptation similarity to host.
- **Biological sanity:** GC distribution versus host, homopolymer lengths, optional predicted folding energy distribution versus real genes.
- **Capacity:** bits per codon and bits per nucleotide versus baselines, with and without ECC overhead.

- **Attack robustness:** survival rate under random mutation, trimming, partial deletion, codon shuffling and re-synonymizing a percentage of codons; report the break-point for each.
- **Pareto view:** stealth versus robustness versus capacity trade-off plot.

12. Tech Stack

- **Frontend:** Next.js (React) on Vercel; charts with a library such as Recharts or Plotly; sequence viewer with simple custom components.
- **Backend / inference API:** Python with FastAPI, hosted on Hugging Face Spaces or Render.
- **ML:** PyTorch for generator, discriminator and decoder; scikit-learn and XGBoost for the k-mer detector baseline.
- **Bio tooling:** Biopython for parsing, translation and codon handling; ViennaRNA for folding energy if included; optional lightweight predictor instead for speed.
- **ECC:** a Reed-Solomon library for Python.
- **Crypto:** Python cryptography library for keyed placement (key-derived pseudorandom selection) and MAC or signature.
- **Database / registry:** Supabase (Postgres) for stamps, owners, timestamps and hashes.
- **Storage of model weights:** Hugging Face Hub or bundled with the Space.
- **Training environment:** Google Colab or Kaggle GPU; small models so training fits free tiers.

13. Deployment Notes

- Keep model inference small (CPU-friendly) so the free tier backend responds quickly.
- Keep sequences to a few hundred codons for the demo.
- Pre-compute the Arena baseline results to keep the live demo responsive.
- Secrets (keys) stay on the backend only; never ship them to the browser.

14. Demo Flow (Approx. 3 to 4 Minutes)

1. Stamp a real E. coli gene with an ID. Show the protein match proof.
2. Run Stealth Check: naive tag flagged with high confidence, GeneStamp scores near chance.
3. Open Stress Lab: raise mutation and indel rates and show the ID still recovering, then show where it finally fails.
4. Run Attacker Arena with a couple of attacks live.
5. Forensic Trace: upload a "stolen" mutated sample and show origin identified with confidence.
6. Close with the registry timeline and the honest limits slide.

15. Scope for a 15-Hour MVP

Must have:

- Keyed synonymous-codon stamping with Reed-Solomon.
- Verify with substitution noise (and basic indel handling).
- Stealth Check with a k-mer detector (naive vs stamped).
- Stress Lab slider.

- Deployed frontend and API.

Should have:

- Attacker Arena with two or three attacks.
- Forensic Trace against a small registry.
- GAN-based realism refinement (can be shown as a trained model with ablation if time allows).

Nice to have:

- Evolution Simulator.
- Held-out CNN detector.
- Leaderboard.

Cut first if short on time: Evolution Simulator, leaderboard, multi-host support.

16. Suggested Team Split (6 People, Adjust to Actual Team)

1. **Coding and crypto:** Reed-Solomon, framing, key-based placement, MAC.
2. **Channel and attacks:** substitution/indel simulator, attack suite for the Arena.
3. **Generative model:** embedder and GAN refiner, training runs.
4. **Detectors:** k-mer classifier, held-out CNN detector, AUC reporting.
5. **Bio constraints and data:** NCBI data prep, codon tables, validity checks, protein identity proof.
6. **Frontend, API and integration:** Next.js app, FastAPI service, registry, deployment.

17. Risks and Honest Limitations

- **Simulation only:** no wet-lab validation; state this upfront and list wet-lab testing as the next milestone.
- **Rewrite attacks:** heavy synonymous recoding can erase a codon-level stamp; the project measures this rather than denying it.
- **Prior art overlap:** closest work is DNAMark and the 2012 gene watermarking paper; related work must cover them and the contribution must be framed as stealth and attack evaluation plus a usable product.
- **Dual-use concern:** natural-looking hidden DNA could also help evade biosecurity screening. Position the product as consent-based traceability designed to work alongside screening, and avoid marketing it as covert or undetectable messaging.
- **Capacity limits:** tags are small by design; this is not a bulk storage system.
- **Training instability:** four-term GAN training can diverge; use the staged schedule and the structural synonymous-codon constraint to reduce risk.

18. Pitch Skeleton

- Problem: proving ownership of engineered biology is hard and tags are easy to strip.
- Existing work: watermarks and signatures exist, mostly as papers or pilots.
- Our contribution: a tag that is tested for stealth and tested under attack, packaged as a usable app.
- Proof: detector AUC near chance, 100% protein match, live attack results.

- Honest limits and next step: wet-lab validation and partnerships with a synthesis provider or lab.

19. Open Decisions

- Final product name (GeneStamp is a placeholder).
- Whether the GAN is core or an ablation-backed refinement for the MVP.
- Which host organisms to include.
- Whether the paper and the product share one repo or are split.
- Which attacks to include in the Arena for the demo.